# A language model, from scratch: every step is a matrix × vector

*Background reading for Parts 2–3. No machine-learning background assumed.*

No background assumed. By the end of this page you will know exactly what a small language model *computes* to write a story — and you will see that almost all of that computation is one operation repeated: **a matrix times a vector.** We finish by showing how running several tokens at once turns that matrix–**vector** product into a matrix–**matrix** product. That single change is the hinge the rest of this project turns on.

Nothing here is about any particular hardware or tool. It is just the arithmetic.

---

## 0.1 What a language model actually does

Here is a story a 15-million-number model wrote:

> *Once upon a time, there was a little girl named Lily. She loved to play outside in the sunshine. One day, she saw a big, red ball in the sky...*

It did **not** write the sentence all at once. It wrote it **one piece at a time**, like this:

```
"Once"  →  model  →  "upon"
"Once upon"  →  model  →  "a"
"Once upon a"  →  model  →  "time"
"Once upon a time"  →  model  →  ","
...
```

Every step, the model looks at all the text **so far** and answers one question: **what is the most likely next piece of text?** It picks one, sticks it on the end, and asks again. That loop — predict next, append, repeat — is the *entire* job. This is called **autoregressive** generation ("auto" = self, "regressive" = feeding its own output back in).

So the whole model is a function:

```
text-so-far   →   [ a score for every possible next piece of text ]
```

and then we pick the highest-scoring piece. Everything below is how that arrow in the middle is computed.

---

## 0.2 Tokens: text is really a list of integers

Computers don't work on letters directly. First the text is chopped into **tokens** — common chunks of text, roughly word-sized (sometimes a whole word, sometimes a piece of one). Each distinct token has an integer id. Our model knows a fixed **vocabulary** of **32000** tokens, so every token is just a number from 0 to 31999.

```
"Once upon a time"   →   [ 9038 ,  2501 ,  263 ,  931 ]     (4 tokens, 4 integers)
```

The model's output — "a score for every possible next token" — is therefore a list of **32000 numbers**, one score per vocabulary entry. The biggest score is the token it thinks comes next. Those 32000 output scores are called the **logits**.

---

## 0.3 Embeddings: turn each token id into a vector of numbers

An integer id like `9038` carries no meaning a calculator can use. So the first thing the model does is look up each token in a big table and replace it with a **vector** of real numbers — a list of, in our model, **288** numbers:

```
token id 9038   →   [ 0.13, -0.42, 0.88, ... , 0.05 ]     (288 numbers)
```

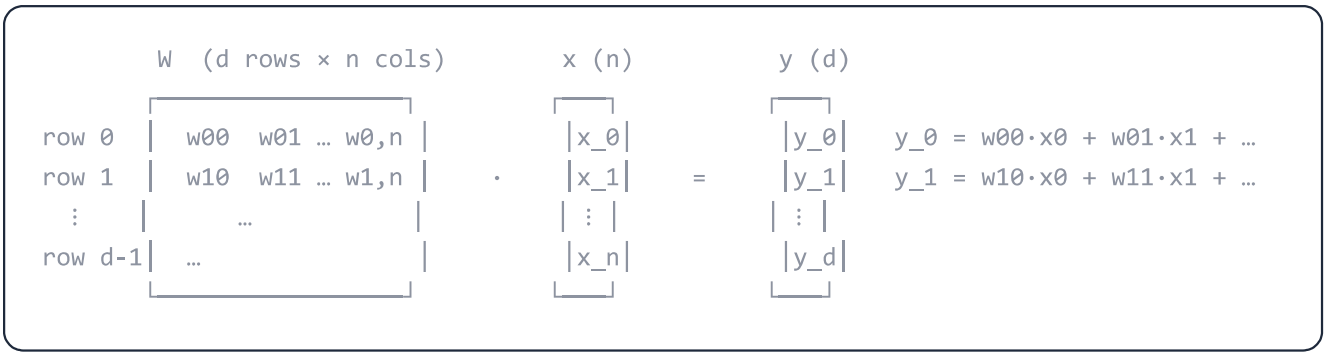
That table (the **embedding matrix**) is just storage: 32000 rows, each row a 288-number vector. Looking up a token is not arithmetic, just "go fetch row 9038."

This 288-number vector is the token's **hidden state**: the model's working representation of "the meaning here, so far." From now on, **the thing flowing through the model is a 288-number vector.** Call this number, 288, the **model width** ( `dim` ). Hold onto that — it is the size of almost everything.

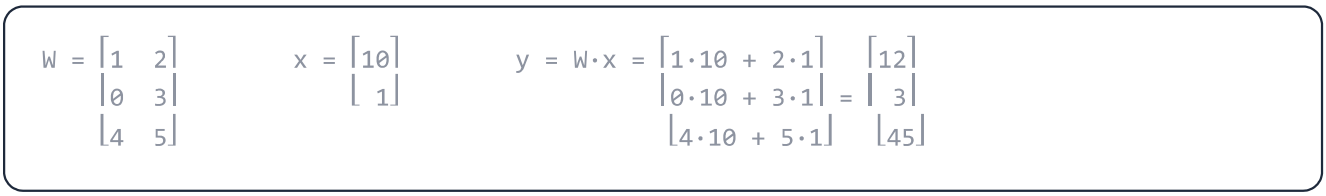
---

## 0.4 The one operation that does all the work: matrix × vector

Before we go further, the star of the show. Suppose you have a **vector** `x` of length `n` , and a **matrix** `W` with `d` rows and `n` columns. The **matrix–vector product** `y = W · x` is a new vector of length `d` , where each entry of `y` is the **dot product** of one row of `W` with `x` :




A tiny concrete example — `d = 3` , `n = 2` :




Three output numbers, each a dot product of a row of `W` with `x` . That's it. In the literature this is called a **GEMV** — **GE**neral **M**atrix–**V**ector multiply.

**Every "learned transformation" in the model is exactly this:** take the current 288-number vector, multiply by a matrix of learned weights, get a new vector. The matrix is what the model "knows"; the multiply is how it "thinks." Different matrices have

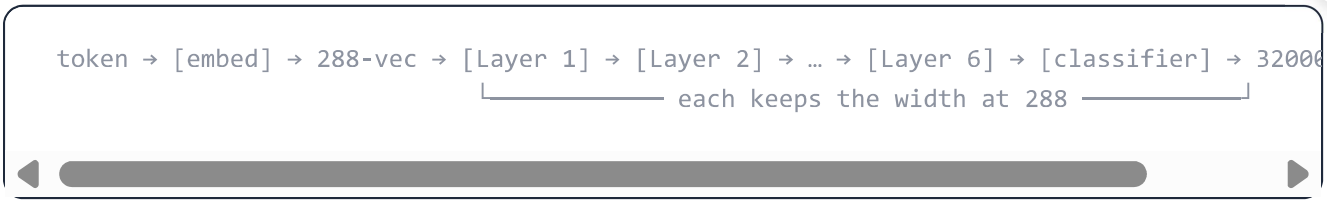
different row counts `d` (they change the vector's length), but the pattern never changes: **output = weight-matrix · input-vector.**

Keep a mental note of the *cost*: a `d × n` matrix–vector product does `d × n` multiply-adds and, crucially, **reads all `d × n` weights to produce just `d` output numbers.** We will come back to that lopsided ratio at the very end.

---

## 0.5 The overall shape: a stack of identical layers

Between the embedding (turns a token into a 288-vector) and the final scores (32000 logits), the model is a **stack of 6 identical layers**. Each layer takes the 288-vector in and produces an improved 288-vector out:




Every layer does the same two things in order: an **attention** block (lets this token gather information from earlier tokens) and a **feed-forward** block (lets the model transform that information). Let's open one layer up. There are only a handful of GEMVs inside, and we'll label each with its matrix shape.

---

## 0.5.1 What actually flows between layers — and what "earlier tokens" means

Two things trip everyone up the first time. Both are worth pinning down before we open a layer, because the single-token walkthrough below can feel like it contradicts the idea that attention "looks at other tokens."

**(a) Each layer has its own weights; the vector just flows on.** Layer 1 outputs an improved 288-vector — and that *is* Layer 2's input. Layer 2 runs the same seven GEMVs, but with its **own separate** matrices ( `Wq…W2` are different learned numbers in every layer), and hands its 288-vector to Layer 3. Six times, then the classifier. The width stays **288 the whole way** — that's *why* layers can stack — and the same vector

is refined six times over. (This running 288-vector is often called the **residual stream**.) So "after the first layer" simply means "into the second layer," repeated.

**(b) "Earlier tokens" means previous steps, remembered separately at each layer.** You are following **only the newest token's** 288-vector down through the 6 layers — the earlier tokens are *not* flowing alongside it. Instead, back when each earlier token was processed, it computed its key `k` and value `v` **at every layer** and saved them in **that layer's cache**. So Layer 1 holds a cache of the layer-1 keys/values of all earlier tokens; Layer 2 holds a *separate* cache of the layer-2 keys/values; and so on — **one cache per layer.** When the current token reaches a layer's attention step, it compares against *that* layer's cache:

```
current token's 288-vector
     │
Layer 1 attention:  its q  vs.  { layer-1 k,v of all earlier tokens }   ← cache₁
     │  (vector refined, still 288 wide)
Layer 2 attention:  its q  vs.  { layer-2 k,v of all earlier tokens }   ← cache₂
     │
    ⋮
Layer 6 attention:  its q  vs.  { layer-6 k,v of all earlier tokens }   ← cache₆
     │
classifier → 32000 logits
```

So attention really is "looking at the other tokens" — but those other tokens live in the per-layer caches, while only the **one** current vector is actually being computed. That is how the model stays a single 288-vector (one token in flight) yet still mixes in the entire history. The very first token has an empty history, so it attends only to itself; by the time we reach *"Once upon a time → ?"*, the new token attends, at each layer, to what "Once", "upon", "a", "time" deposited in that layer earlier.

With that settled, here is what one layer does to the vector.

---

## 0.6 Inside one layer — part A: attention

Attention is how a token looks back at the earlier tokens and pulls in what's relevant ("the ball" should pay attention to "red" and "sky"). It works in four steps.

**Step 1 — make three vectors from the current one (three GEMVs).** The model multiplies the 288-vector by three different learned matrices to get a **query**, a **key**, and a **value**, each also 288 numbers:

```
q = Wq · x        Wq is 288 × 288      (GEMV: 288 → 288)
k = Wk · x        Wk is 288 × 288      (GEMV: 288 → 288)
v = Wv · x        Wv is 288 × 288      (GEMV: 288 → 288)
```

Intuition: the **query** is "what am I looking for?", the **key** is "what do I offer?", the **value** is "what I'll hand over if you pick me."

**Step 2 — remember k and v.** The `k` and `v` for this token are saved in a cache, so future tokens can look back at it. (This cache is why the model doesn't recompute the whole past every step.)

**Step 3 — score and combine (dot products, no learned matrix).** The token compares its query `q` against the key of **every earlier token** with a dot product — a high score means "that token is relevant to me." The scores are squashed into weights that add up to 1 (a **softmax**), and the output is the weighted average of all the earlier **value** vectors. This step's size depends on how many tokens came before, not on a fixed weight matrix — so it's usually small compared to the GEMVs, and it uses no learned matrix. (There's a small position-encoding tweak called RoPE applied to `q` and `k` first; it's cheap and you can ignore it for the cost story.)

**Step 4 — project the result back (one GEMV).** The combined vector is multiplied by one more learned matrix to produce the block's output:

```
attn_out = Wo · (combined)      Wo is 288 × 288     (GEMV: 288 → 288)
```

Finally the layer **adds** this back onto the vector it started with (a "residual" connection — just element-wise addition, cheap):

```
x  =  x  +  attn_out
```

**Attention's tally: 4 GEMVs** ( `Wq`, `Wk`, `Wv`, `Wo` , each 288 → 288), plus the cheap score-and-combine step.

## 0.7 Inside one layer — part B: the feed-forward network (FFN)

The second block transforms each token's vector on its own. It briefly expands the vector to a **wider** size (768, the *hidden* width), does a cheap non-linear step, then shrinks it back to 288. Three GEMVs:

```
a = W1 · x        W1 is 768 × 288      (GEMV: 288 → 768)   ← widen
g = W3 · x        W3 is 768 × 288      (GEMV: 288 → 768)   ← widen (a "gate")
h = silu(a) * g                        (cheap, element-wise: a gate that lets some
                                        numbers through and damps others)
y = W2 · h        W2 is 288 × 768      (GEMV: 768 → 288)   ← shrink back
```

Then another residual add: `x = x + y`. Intuition: attention *mixes information between tokens*; the FFN *does per-token processing* — it's where a lot of the model's "knowledge" lives. (There are also two cheap normalization steps per layer, called RMSNorm, one before each block. They rescale the vector; no matrix multiply worth counting.)

**FFN's tally: 3 GEMVs** ( `W1, W3` at 288 → 768, and `W2` at 768 → 288).

So **one full layer = 4 + 3 = 7 GEMVs.** Six layers = **42 GEMVs**.

---

## 0.8 The finish: scoring every possible next token (one big GEMV)

After the 6 layers, we have a final 288-number vector that summarizes "everything the model understands about what should come next." One last matrix turns it into a score for **each** of the 32000 vocabulary tokens:

```
logits = Wcls · x     Wcls is 32000 × 288    (GEMV: 288 → 32000)
```

This is the **biggest** matrix in the whole model — 32000 rows! — because the vocabulary is large. The 32000 output numbers are the scores. Pick the token with the

highest score (or sample one at random in proportion to the scores, for variety), append it to the text, and **go back to 0.1** for the next token.

---

## 0.9 Add it up — a token is 43 matrix–vector products

Producing **one** token runs the whole network once:

| Where | Matrix | Shape ( `out ← in` ) | How many |
|---|---|---|---|
| attention | `Wq, Wk, Wv, Wo` | 288 ← 288 | 4 per layer |
| FFN | `W1, W3` | 768 ← 288 | 2 per layer |
| FFN | `W2` | 288 ← 768 | 1 per layer |
| **per layer** | | | **7** |
| **× 6 layers** | | | **42** |
| classifier | `Wcls` | 32000 ← 288 | **1** |
| **total per token** | | | **43** |

**43 matrix–vector products per token, and essentially nothing else that costs anything.** RMSNorm, the softmax, RoPE, the residual adds — all cheap element-wise work. If you wanted to make this model faster, you would make **matrix × vector** faster. Full stop.

And notice **why it's a matrix × *vector*** and not something bigger: the model generates **one token at a time**, so at each step exactly **one** 288-number vector is flowing through. One vector in, one vector out, at every matrix. That "1" is about to become the whole story.

---

## 0.10 From GEMV to GEMM: do many vectors at once

Here is the lopsided fact from §0.4, made concrete on the classifier: `Wcls` has `32000 × 288 ≈ 9.2 million` weights. A single token's classifier GEMV **reads all 9.2 million**

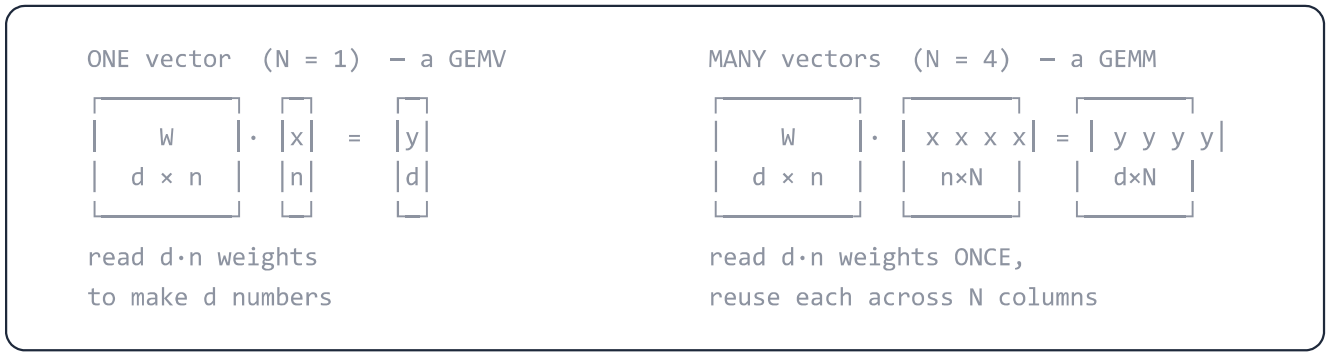
**weights** to produce **32000** output numbers — then throws the weights away and, next token, reads all 9.2 million again. You are spending almost all your effort *fetching weights*, and using each one for a single multiply.

There is an obvious fix: **multiply the same matrix by several vectors at once.** If you have `N` different input vectors that all need to go through `Wcls`, stack them side by side as the **columns** of a matrix `X` (shape `288 × N`). Then

```
Wcls · X   =   Y          (32000×288) · (288×N)  =  (32000×N)
```

produces all `N` results in one shot — and now each weight, fetched once, is used `N` times (once per column). This is a matrix–**matrix** product: a **GEMM** (**GE**neral **M**atrix–**M**atrix multiply). The GEMV you've seen all along is just GEMM with `N = 1`.

Picture the difference:




The arithmetic per column is identical; what changes is that the **cost of reading the weights is now shared across `N` results.** That is why GEMM is the operation big, efficient systems are built around, and GEMV (N = 1) is the wasteful special case.

### Where do the extra vectors come from?

Two natural sources, both real:

- **Prefill — the prompt.** When you hand the model a prompt like *"Once upon a time"* (4 tokens), all 4 tokens are already known up front. You don't need them one at a time — you can push all **4 vectors through every matrix together**: `N = 4`, a GEMM. (Only *after* the prompt, when the model starts inventing new tokens, is it forced back to one-at-a-time.)
- **Batching — many stories at once.** Generate `B` independent stories in parallel. At each step you have `B` current vectors (one per story), so every matrix multiply is `N`

`= B`. You produce `B` tokens per step instead of 1 — the same weight-reads, `B×` the useful output.

In both cases the model's math is *unchanged*: the same 43 matrices, the same dot products. You have only changed **how many vectors ride through at once** — turning `N = 1` GEMVs into `N > 1` GEMMs, and turning "read the weights, use them once" into "read the weights, use them `N` times."

---

## 0.11 What you now know

1. A language model writes text **one token at a time**: look at everything so far, score all 32000 possible next tokens, pick one, repeat.
2. The text becomes integers (**tokens**), each integer becomes a **288-number vector** (an **embedding**), and that vector flows through **6 layers** and a final **classifier**.
3. Nearly all the work is **matrix × vector** (**GEMV**): each learned matrix turns the current vector into a new one. There are **43 of them per token** (7 per layer × 6, plus the classifier), and everything else is cheap.
4. It's a *vector* (not a matrix) because the model runs **one token at a time**: `N = 1`.
5. Stacking several vectors side by side (from **prefill** or **batching**) makes the input a matrix and turns each GEMV into a **GEMM** (`N > 1`) — which lets every weight, read once, do `N` multiplies instead of one.

That's the whole model, and the whole reason GEMM matters. From here, the natural next questions are *how big are these matrices really, where does the time actually go,* and *how much does turning GEMV into GEMM buy you* — which is where the rest of the story picks up.